



JERARQUÍA FHS

FILESYSTEM HIERARCHY STANDARD



¿QUÉ ES EL FHS?

El **Filesystem Hierarchy Standard** (FHS) define la estructura de directorios y contenidos en sistemas Unix-like.



Estandariza la ubicación de archivos



Asegura compatibilidad entre sistemas



Facilita la administración y desarrollo



ORGANIZACIÓN

Jerarquía Lógica y Predecible

EVOLUCIÓN DESDE UNIX



1970s

Orígenes en **Unix**
Estructura simple y monolítica



1980s

Divergencia **BSD / System V**
Incompatibilidad creciente



1993

Proyecto **FSSTND**
Primer intento de
estandarización Linux



1996+

Nace el **FHS**
Estándar universal actual

DIRECTORIO RAÍZ (/)

Origen de la Jerarquía

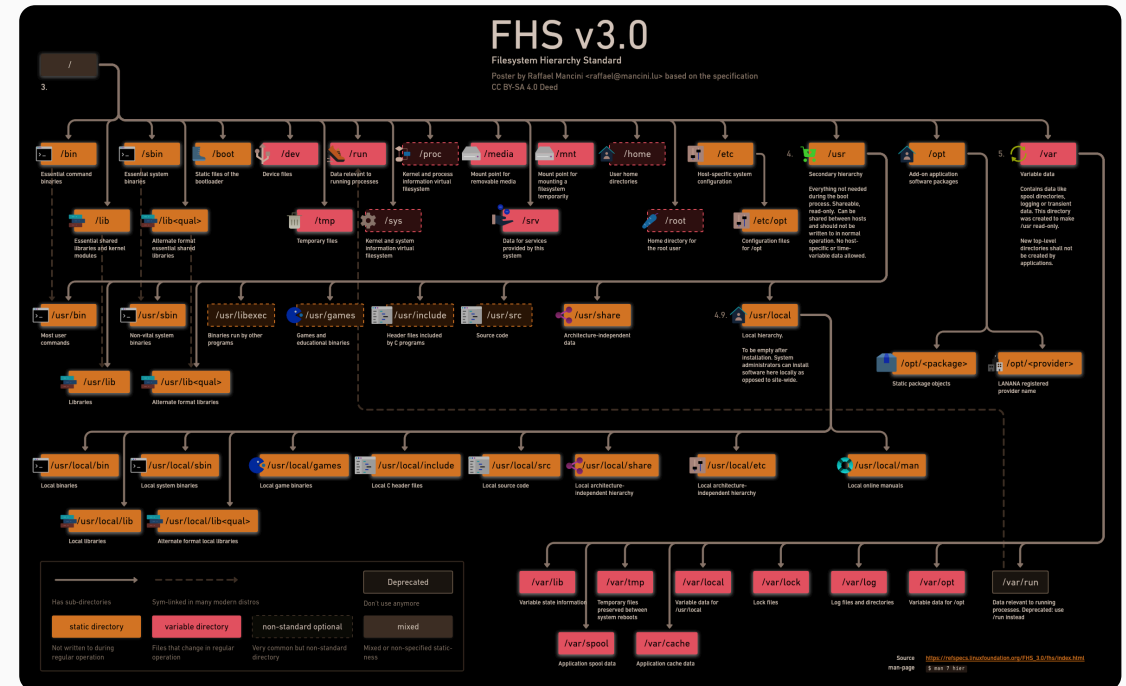
El directorio / (raíz) es el nivel más alto. Todos los archivos y directorios existen bajo esta estructura.

Árbol Invertido

La estructura se visualiza como un árbol donde la raíz está arriba y las ramas (subdirectorios) descienden.

Punto de Montaje

Todo sistema de archivos se monta bajo /, unificando discos y particiones en una sola jerarquía lógica.



DIRECTORIOS ESENCIALES (I)

/BIN

Binarios esenciales de usuario. Comandos disponibles para todos los usuarios en modo single-user.

Ejemplos: **ls, cp, cat, echo**

USUARIO

/SBIN

Binarios de administración del sistema. Utilidades para mantenimiento y configuración del sistema.

Ejemplos: **fdisk, reboot, ifconfig**

ADMIN

/ETC

Archivos de configuración específicos del host. No debe contener binarios ejecutables.

Ejemplos: **passwd, fstab, hosts**

CONFIG

/DEV

Archivos de dispositivos. Puntos de acceso a hardware y periféricos del sistema.

Ejemplos: **sda, null, random, tty**

DISPOSITIVOS

DIRECTORIOS ESENCIALES (II)

/USR

Jerarquía secundaria. Programas y datos de usuario de solo lectura.

Contiene: **bin, lib, share**

PROGRAMAS

/VAR

Datos variables. Archivos que crecen durante la operación del sistema.

Contiene: **log, spool, mail**

VARIABLES

/HOME

Directorios personales de los usuarios. Almacén de datos y configuraciones individuales.

Ejemplo: **/home/usuario**

PERSONAL

/TMP

Archivos temporales. Borrados automáticamente al reiniciar o periódicamente.

Permisos: **Sticky bit activo**

TEMPORAL

SISTEMAS DE FICHEROS EN MEMORIA



/tmp (tmpfs)

Almacenamiento temporal basado en **RAM**. Rápido acceso, volátil.



/dev/shm

Memoria compartida. Comunicación eficiente entre procesos.



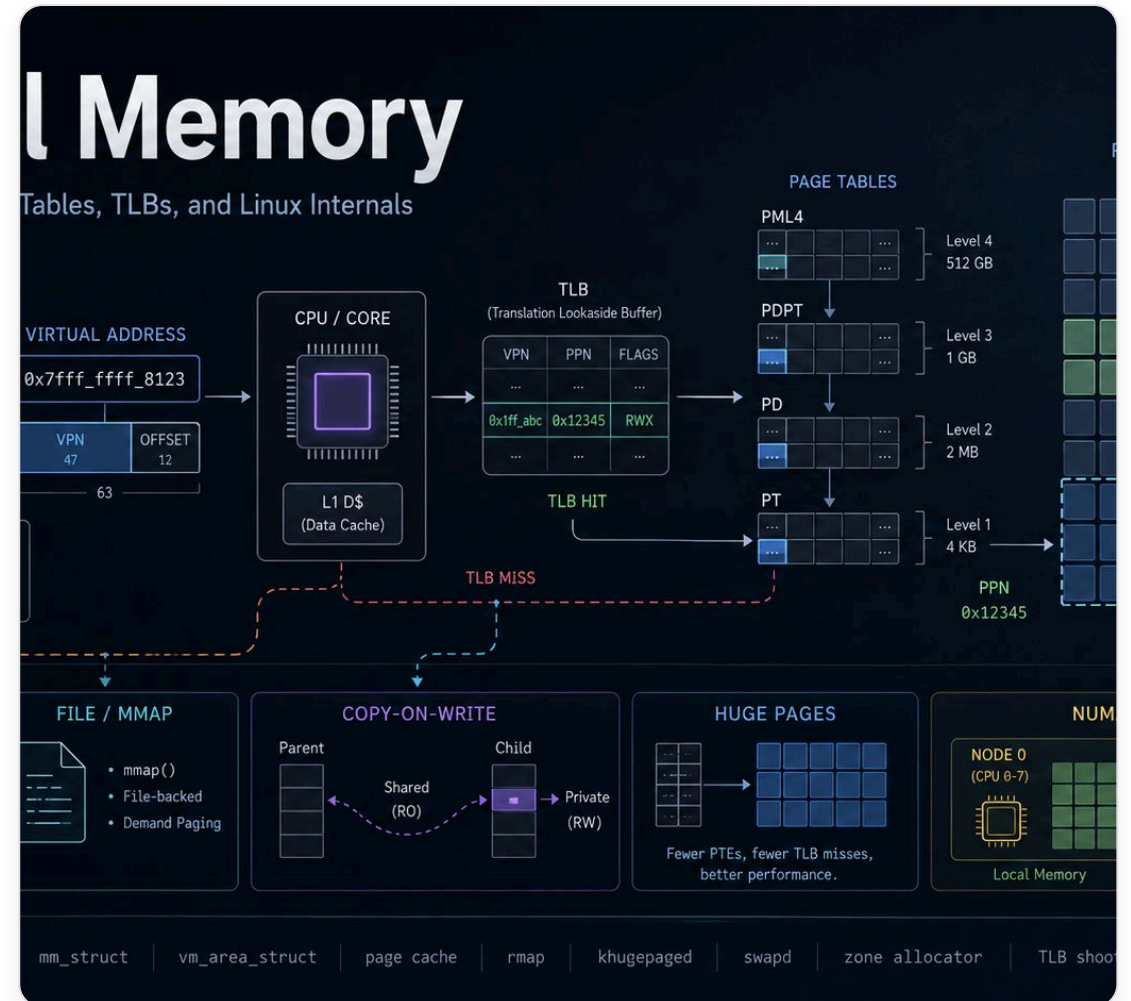
/run

Datos de tiempo de ejecución. Sockets, PIDs. **Volátil** al reiniciar.



/proc & /sys

Sistemas virtuales. Info del kernel y configuración de dispositivos.



GESTIÓN DE DIRECTORIOS Y FICHEROS (I)

TOUCH

Propósito principal: Cambiar marca de tiempo.

Efecto secundario: Crea archivo si no existe.

```
$ touch archivo.txt # Crea archivo vacío
$ touch -c existente.txt # No crea si existe
$ touch -t 202601011200 archivo # Cambia
timestamp
```

Nota: Ideal para actualizar tiempos de acceso/modificación sin editar el archivo.

MKDIR

Make Directory. Crea nuevos directorios.

Opción clave: **-p** (parents).

```
$ mkdir carpeta # Crea un directorio
$ mkdir -p dir1/dir2/dir3 # Crea estructura
anidada
$ mkdir -m 755 seguro # Con permisos
específicos
```

Consejo: Usa **-p** para evitar errores si el directorio padre no existe.

GESTIÓN DE DIRECTORIOS Y FICHEROS (II)

RMDIR

Elimina directorios **vacíos**. Para eliminar con contenido, usar `rm -r`.

```
$ rmdir carpeta_vacia
$ rmdir -p dir1/dir2 # Elimina estructura
```

RM

Elimina archivos o directorios. **¡Peligroso!** Usar con precaución.

```
$ rm archivo.txt
$ rm -r directorio # Recursivo
$ rm -rf dir # Forzado y recursivo
```

CP

Copia archivos o directorios. Usa **-r** para directorios.

```
$ cp origen.txt destino.txt
$ cp -r dir_origen/ dir_destino/
$ cp -a dir/ backup/ # Archivo completo
```

MV

Mueve o **renombra** archivos y directorios.

```
$ mv viejo.txt nuevo.txt # Renombrar
$ mv archivo /destino/ # Mover
$ mv -i a.txt b.txt # Pregunta si sobrescribe
```

COMANDOS DE INFORMACIÓN DEL SISTEMA



TOP / HTOP

Muestra procesos en tiempo real. **htop** ofrece interfaz mejorada e interactiva.

```
$ top  
$ htop
```



DF

Espacio libre en disco. Usa **-h** para formato legible (Human-readable).

```
$ df -h
```



FREE

Muestra uso de memoria RAM y Swap. Opción **-h** para unidades legibles.

```
$ free -h
```



UNAME

Información del kernel. **-a** muestra todos los detalles del sistema.

```
$ uname -a
```

PROFUNDIZANDO EN EL SISTEMA

LSBLK

Lista dispositivos de bloque (discos, particiones).
Muestra la **topología** de almacenamiento de forma clara.

/PROC

Sistema de archivos virtual. Lee **cpuinfo** y **meminfo** para detalles de hardware en tiempo real.

DMESG

Muestra mensajes del **kernel**. Útil para diagnosticar errores de hardware y proceso de arranque.



```
$ lsblk
```

```
NAME MAJ:MIN RM SIZE RO TYPE MOUNTPOINT
```

```
sda 8:0 0 500G 0 disk
```

```
└─sda1 8:1 0 512M 0 part /boot
```

```
└─sda2 8:2 0 499.5G 0 part /
```

```
$ cat /proc/cpuinfo # Info del procesador
```

```
$ cat /proc/meminfo # Info de memoria
```

```
$ dmesg | grep "error" # Filtrar errores
```

CONCLUSIÓN



ORGANIZACIÓN ESTANDARIZADA

El FHS garantiza la **previsibilidad** y compatibilidad en sistemas Unix-like, facilitando la administración y el desarrollo.



JERARQUÍA CLARA

Separación lógica entre directorios de **administración** (/sbin, /etc) y de **usuario** (/bin, /usr).



HERRAMIENTAS ESENCIALES

Dominar comandos de gestión e información es clave para un **control total** del sistema y su diagnóstico.



FHS

La base de la estructura Unix